

Design by Contract

Rael Cassimiro

O artigo sobre Design by Contract apresenta uma abordagem voltada para aumentar a confiabilidade, robustez e previsibilidade no desenvolvimento de software por meio da definição explícita de contratos entre os componentes de um sistema. Bertrand Meyer propõe que toda rotina ou módulo possua responsabilidades claramente definidas, estabelecendo obrigações e garantias tanto para quem utiliza quanto para quem implementa determinada funcionalidade. Essa visão busca reduzir ambiguidades, facilitar manutenção e tornar os sistemas mais seguros e consistentes, principalmente em ambientes complexos e orientados à reutilização de componentes.

A principal ideia do artigo é que o software deve funcionar como um conjunto de acordos formais. Assim como em contratos do mundo real, cada parte possui deveres e direitos bem estabelecidos. No contexto computacional, isso ocorre por meio das pré-condições, pós-condições e invariantes. As pré-condições representam aquilo que deve ser garantido antes da execução de uma operação. As pós-condições definem os resultados esperados após a execução. Já as invariantes asseguram que determinados estados do objeto permaneçam válidos durante todo seu ciclo de vida. Essa estrutura fornece maior clareza sobre o comportamento esperado do sistema e reduz a necessidade de verificações excessivas espalhadas pelo código.

Outro aspecto importante discutido pelo autor é a crítica ao chamado defensive programming. Meyer argumenta que um sistema excessivamente defensivo aumenta complexidade e dificulta manutenção, pois cria redundâncias e responsabilidades confusas entre os componentes. O Design by Contract propõe justamente o contrário: cada parte do sistema deve responder apenas pelas obrigações que lhe cabem. Caso uma pré-condição não seja atendida, por exemplo, o erro pertence ao cliente da rotina, não ao fornecedor. Isso torna mais simples identificar responsabilidades e localizar falhas durante o desenvolvimento.

O artigo também dedica grande atenção ao tratamento de exceções. Meyer afirma que existem apenas duas respostas válidas diante de uma falha: recuperar o sistema e tentar novamente a operação ou interromper a execução reportando claramente o problema ao chamador. Essa abordagem evita que falhas sejam ocultadas e garante maior previsibilidade ao comportamento da aplicação. O autor chama atenção para o fato de que uma rotina nunca deve aparentar sucesso caso não tenha conseguido cumprir seu contrato. Essa visão é extremamente relevante em sistemas modernos que dependem de integrações, comunicação distribuída e processamento contínuo de dados.

Ao comparar os conceitos do artigo com minhas experiências profissionais, percebo que muitos princípios do Design by Contract se relacionam diretamente com atividades que desenvolvi em projetos de dados, BI e integração de sistemas. Em minha trajetória profissional atuei em empresas como TOTVS, Matrix Go e 90 Tecnologia da Informação, trabalhando com desenvolvimento de consultas SQL, integração de APIs, automação de relatórios, modelagem

de dados, dashboards em Power BI e processos de ETL. Nessas experiências, a necessidade de garantir consistência e confiabilidade dos dados sempre esteve presente, tornando os conceitos do artigo extremamente aplicáveis ao contexto real de trabalho.

Na atuação atual como Analista BI, desenvolvendo dashboards e processos de ETL, os contratos aparecem constantemente de forma implícita. Em pipelines de dados, por exemplo, determinadas regras precisam ser respeitadas antes da execução das transformações. Uma tabela de origem deve conter campos obrigatórios, os dados precisam seguir padrões específicos e determinadas integrações dependem de estruturas válidas para funcionar corretamente. Essas validações representam claramente pré-condições. Após o processamento, espera-se que os dados sejam carregados corretamente, sem duplicidade, inconsistências ou falhas de relacionamento, o que corresponde às pós-condições descritas por Meyer.

Em projetos de Business Intelligence, especialmente utilizando Power BI, SQL Server e Oracle, também observei situações em que contratos bem definidos poderiam evitar diversos problemas operacionais. Em muitos ambientes corporativos, relatórios dependem de múltiplas fontes de dados e integrações entre sistemas distintos. Quando não existe uma definição clara das responsabilidades de cada módulo, pequenas alterações em tabelas ou APIs acabam quebrando dashboards e indicadores estratégicos. O conceito de contrato ajudaria justamente a reduzir esse tipo de impacto, definindo exatamente quais dados são esperados, quais formatos devem ser mantidos e quais garantias cada sistema deve fornecer.

Durante minha experiência na TOTVS, trabalhei no desenvolvimento e manutenção de relatórios, cubos OLAP e dashboards integrados aos sistemas RM, SmartView e Protheus. Também participei da integração de APIs e homologação técnica de entregas voltadas para performance e integridade dos dados. Nesse contexto, a ideia de Design by Contract poderia ser aplicada para formalizar regras entre os módulos de integração e os sistemas consumidores, reduzindo falhas de comunicação e aumentando previsibilidade nas entregas. Além disso, contratos explícitos ajudariam a documentar melhor requisitos técnicos e comportamentos esperados das integrações.

Outro ponto importante está relacionado ao tratamento de exceções em processos automatizados. Em pipelines ETL, erros podem ocorrer por falhas de conexão, dados inválidos, indisponibilidade de APIs ou inconsistências estruturais. Na prática, muitos sistemas tentam executar novamente determinadas etapas após falhas temporárias, comportamento muito semelhante ao conceito de resumption apresentado por Meyer. Quando a recuperação não é possível, o processo interrompe a execução e registra logs detalhados para análise posterior, seguindo exatamente a ideia de falha organizada defendida pelo autor.

A filosofia do Design by Contract também possui relação direta com arquiteturas modernas orientadas a serviços e APIs REST, áreas nas quais também tive contato profissional. APIs funcionam naturalmente através de contratos definidos entre cliente e servidor. Um endpoint estabelece quais parâmetros devem ser enviados, quais respostas serão retornadas e quais condições precisam ser atendidas para execução correta da operação. Esse modelo é

extremamente próximo da proposta do artigo e demonstra como os conceitos de Meyer continuam atuais mesmo décadas após sua criação.

Além dos benefícios técnicos, o artigo também demonstra impacto organizacional importante. Contratos claros facilitam comunicação entre equipes, melhoram documentação e tornam processos de manutenção mais eficientes. Em ambientes corporativos de dados, onde múltiplos setores dependem das mesmas informações para tomada de decisão, essa clareza reduz retrabalho, inconsistências e falhas de interpretação de regras de negócio.

Portanto, considero que o principal mérito do artigo está em mostrar que confiabilidade não deve ser tratada apenas como etapa de testes, mas incorporada diretamente na arquitetura e no desenvolvimento do software. O Design by Contract fornece uma abordagem sólida para construção de sistemas mais previsíveis, reutilizáveis e robustos. Ao relacionar esses conceitos com minha experiência profissional em BI, SQL, ETL, integração de APIs e análise de dados, fica evidente que os princípios apresentados possuem grande aplicabilidade prática e continuam extremamente relevantes para os desafios atuais da engenharia de software e da engenharia de dados.